NETWATCH

Real-Time Network Monitoring, Performance Analysis and Anomaly Detection System

IDE Execution Blueprint

Requirements • Architecture • Database/API • Roadmap • Testing • Runbook

Designed for a lightweight laptop-based implementation using original real-time telemetry.

Document Map

Document	Purpose
PRD	Product scope, users, priorities, success criteria
SRS	Functional/non-functional requirements
TRD	Technology choices and technical design
Architecture	Repository structure, modules, execution flow
Database/API	Data model and backend contracts
Roadmap	Phase-by-phase implementation with exit gates
Testing	Automated/manual acceptance and Definition of Done
Runbook	Environment setup, run commands and troubleshooting
IDE Guide	Ticket backlog for incremental implementation
Report/Demo	Academic report structure and demonstration evidence

Recommended first implementation target: Phases 0-8 only. Treat discovery, packet capture and ML as extensions after the base monitor is stable.

PRD

1. Product name

NetWatch: Real-Time Network Monitoring, Performance Analysis and Anomaly Detection System

2. Product vision

NetWatch is a laptop-friendly real-time network monitoring, performance analysis, alerting, and optional anomaly-detection platform. It collects original telemetry from the host machine and authorized local-network devices, stores time-series measurements, exposes them through a FastAPI backend, and visualizes current and historical network health in a browser dashboard.

The product should behave like a lightweight Network Operations Center (NOC) for a student or small lab environment. It must provide useful live observations using data gathered directly from the monitored environment rather than depending on a static third-party dataset.

3. Problem statement

Small networks often lack a simple, low-cost way to observe device availability, latency, packet loss, network-I/O behavior, host resource usage, and abnormal changes in one interface. Existing enterprise monitoring platforms can be too heavy or complex for a student laptop and for an academic demonstration.

NetWatch addresses this by providing a modular monitor that runs locally, stores its own telemetry, and gives the user current status, history, alerts, and explainable health indicators.

4. Primary users

- Student/developer running NetWatch on a personal laptop.
- Small lab administrator monitoring a few authorized devices.
- Project evaluator/instructor viewing the dashboard and generated reports.

5. Goals

1. Collect original telemetry from the host in near real time.

2. Monitor configured devices for reachability, latency, and packet loss.

3. Store time-stamped metrics for historical analysis.

4. Show a clear web dashboard with live and historical views.

5. Generate alerts when defined thresholds are breached.

6. Remain usable on an 8 GB RAM laptop without a GPU.

7. Provide a clean extension path to discovery, packet analysis, SNMP, and ML anomaly detection.

6. Non-goals for the MVP

- Enterprise-scale monitoring of hundreds or thousands of devices.
- Full packet payload inspection.
- Intrusion prevention or automated blocking.

- Unauthorized scanning of public/external systems.
- Deep-learning models.
- High-availability distributed deployment.

7. Core user stories

- As a user, I want to see whether each configured device is online so I can identify outages quickly.
- As a user, I want to see current and historical latency so I can identify degradation.
- As a user, I want packet-loss measurements so I can distinguish delay from connectivity instability.
- As a user, I want to see my laptop's CPU, memory, disk, upload, and download behavior alongside network metrics.
- As a user, I want live charts that update without manually refreshing the page.
- As a user, I want threshold-based alerts with severity, timestamp, source device, and acknowledgement status.
- As a user, I want historical data so I can compare current values against past behavior.
- As a user, I want the system to calculate a simple network-health score and explain why it changed.
- As an advanced user, I want anomaly detection based on previously collected telemetry.

8. MVP feature set

P0 - must have

- Local host metrics collection.
- Configured-device reachability checks.
- Latency monitoring.
- Packet-loss calculation.
- Network byte/packet counters and rate calculation.
- SQLite database.
- REST API.
- Live WebSocket stream.
- Dashboard overview.
- Device detail view.
- Alert generation and acknowledgement.
- Historical chart queries.
- Configurable polling intervals.
- Logging and graceful error handling.

P1 - should have

- Automatic LAN discovery.
- Hostname/MAC display when available.
- Protocol counters from packet capture.
- Health score.
- CSV export.
- Configurable alert rules through settings.

P2 - could have

- SNMP collectors.
- Port/service monitor for owned/authorized hosts.

- Isolation Forest anomaly detection.
- Multi-agent monitoring.
- PDF report generation.

9. Success metrics

The MVP is successful when:

- dashboard live values refresh within 5 seconds of collection under normal local conditions;
- configured hosts transition between online/offline states correctly in test scenarios;
- at least 24 hours of metrics can be stored and queried without manual cleanup;
- common dashboard API responses are under 500 ms on the development laptop for a small local dataset;
- threshold alerts appear once per incident rather than spamming duplicate alerts each polling cycle;
- application recovers from an unreachable device, missing permission, or collector failure without crashing;
- automated unit/integration tests cover the critical collection, persistence, alert, and API flows.

10. Constraints

- Development target: a personal laptop, approximately 8 GB RAM.
- Default database: SQLite.
- Default scope: local machine and devices the user owns or has permission to monitor.
- Some capabilities such as raw packet capture/ARP discovery may require Administrator/root privileges and, on Windows, Npcap.
- The base application must continue working when optional privileged collectors are disabled.

11. Product risks and mitigations

Risk	Impact	Mitigation
Raw packet capture needs privileges	Feature unavailable	Keep packet capture optional; core metrics use non-privileged collectors
Ping blocked by a device	False offline state	Mark status as unknown/unreachable; permit TCP/service probes later
Too-frequent polling	CPU/battery overhead	Default to 5-10 second local metrics and 10-30 second device probes
Database grows continuously	Disk usage	Add retention settings and aggregation/cleanup job
Duplicate alerts	Poor usability	Track alert state, cooldown, first_seen and last_seen
ML gives misleading alarms	Low trust	ML remains optional and never replaces deterministic thresholds

12. MVP release definition

Version 1.0 is ready when P0 features pass the acceptance tests in 07_TESTING_AND_ACCEPTANCE.md, setup can be reproduced from a clean environment, and a 10-15 minute demonstration can be completed without editing the database manually.

SRS

1. System scope

NetWatch is a local web application comprising telemetry collectors, a scheduler, persistence layer, alert engine, REST/WebSocket API, and browser-based dashboard.

2. Actors

- Operator: runs the application, adds devices, views metrics, manages alerts.
- Monitored host: the laptop on which NetWatch runs.
- Remote authorized device: another LAN host/router/server that NetWatch probes or reads through supported protocols.
- Collector: software component that gathers one type of measurement.
- Dashboard client: browser consuming API and WebSocket data.

3. Functional requirements

FR-001 Host system metrics

The system shall collect CPU percentage, memory usage, disk usage, interface byte counters, packet counters, and application uptime from the local host.

FR-002 Network rate calculation

The system shall calculate upload/download rates from counter deltas over a measured time interval rather than treating cumulative byte counters as instantaneous bandwidth.

FR-003 Device registry

The system shall allow configured devices to be stored with name, IP address, optional hostname/MAC, enabled flag, tags, and monitoring interval.

FR-004 Reachability

The system shall probe enabled devices and record online, offline, or unknown with timestamp and diagnostic detail.

FR-005 Latency

For successful probes, the system shall record round-trip latency in milliseconds.

FR-006 Packet loss

The system shall calculate packet loss from a small configurable set of probes and store loss percentage.

FR-007 Time-series persistence

The system shall persist measurements with UTC timestamps and associated device/interface identifiers.

FR-008 Live dashboard feed

The backend shall publish the latest summary to connected dashboard clients through WebSockets at a configurable cadence.

FR-009 Historical queries

The API shall return time-bounded metric series for a device and metric type.

FR-010 Alerts

The system shall evaluate configurable deterministic rules such as device offline, latency above threshold, packet loss above threshold, CPU above threshold, and memory above threshold.

FR-011 Alert lifecycle

Alerts shall include severity, status, first_seen, last_seen, occurrence_count, source, rule, message, and optional acknowledgement time.

FR-012 Alert deduplication

While a rule condition remains true, the system shall update the active incident rather than insert a new independent alert at every poll.

FR-013 Dashboard overview

The dashboard shall display device counts, online/offline summary, active alerts, host CPU/RAM, current up/down rates, network health score, and recent metric charts.

FR-014 Device detail

The dashboard shall display a selected device's latest status, latency, packet loss, recent history, and alerts.

FR-015 Configuration

Polling intervals, retention days, alert thresholds, and optional collectors shall be configurable through environment variables and/or persisted application settings.

FR-016 Data export

The system should allow selected historical metrics to be exported as CSV.

FR-017 LAN discovery (P1)

When explicitly initiated, the system should discover devices on the user's authorized local subnet using ARP where technically supported.

FR-018 Packet/protocol metrics (P1)

When privileged packet capture is enabled, the system should count protocols, source/destination endpoints, and packet sizes without storing packet payloads by default.

FR-019 Anomaly detection (P2)

The system may train an Isolation Forest on selected historical numeric features and produce anomaly scores for new observations.

4. Non-functional requirements

NFR-001 Performance

For a small network of up to approximately 20 configured devices, normal monitoring shall not make the laptop unusable. The initial target is under roughly 10% sustained CPU attributable to NetWatch during normal polling, excluding intentional packet-capture stress tests.

NFR-002 Memory

The base application should remain comfortably below 1 GB RAM during normal use. Large historical queries shall be bounded/paginated.

NFR-003 Reliability

A failing collector shall be isolated and logged; it shall not terminate the whole scheduler.

NFR-004 Security

The API shall bind to localhost by default. If exposed to the LAN, authentication and explicit host configuration must be added before treating the deployment as trusted.

NFR-005 Privacy

Raw packet payloads shall not be persisted by default. IP/MAC information is treated as local operational data and should remain on the user's device unless deliberately exported.

NFR-006 Portability

The core system shall support Windows and Linux where possible. Privileged features may have OS-specific setup instructions.

NFR-007 Maintainability

Collectors, alert rules, API routes, and persistence services shall be modular rather than implemented inside a single file.

NFR-008 Testability

Time, network probes, and system-metric readers shall be abstractable/mockable for deterministic tests.

NFR-009 Observability

Application logs shall include timestamp, level, module, event, and error details. Sensitive packet payloads or credentials shall never be logged.

NFR-010 Data integrity

All persisted timestamps shall use UTC; the frontend may convert them to local time for presentation.

5. Data requirements

Minimum persisted entities:

- Device
- DeviceStatusMetric
- HostMetric
- NetworkIOMetric
- AlertRule
- Alert
- AppSetting

Advanced entities:

- DiscoveryObservation
- ProtocolMetric
- AnomalyScore

6. Interface requirements

Browser UI

Responsive layout suitable for a laptop screen, with sidebar/top navigation and charts that remain readable at 1366x768.

REST API

JSON over HTTP under /api/v1.

WebSocket

Live dashboard stream under /ws/live.

Database

SQLite file under a configurable data directory.

7. Failure behavior

- A timed-out probe is recorded with failure reason.
- A missing network permission disables only the affected optional collector.
- Database errors are logged and API returns a structured error rather than a stack trace.
- Invalid device addresses are rejected by validation.
- WebSocket clients may reconnect without restarting backend collectors.

8. Acceptance linkage

Each FR/NFR should map to one or more tests in 07_TESTING_AND_ACCEPTANCE.md. P0 requirements are release blockers for MVP unless explicitly waived in the project log.

TRD

1. Design strategy

Use a modular monolith. One Python application is easier to run on a student laptop, while clear internal boundaries preserve the option to split services later.

2. Recommended stack

Layer	Choice	Reason
Runtime	Python 3.11+	Mature networking/system ecosystem
Web framework	FastAPI	Typed APIs, WebSocket support, automatic OpenAPI docs
ASGI server	Uvicorn	Lightweight local development server
ORM	SQLAlchemy 2.x	Explicit models and migrations path
Database	SQLite	Zero-service local persistence
Validation	Pydantic	Input/config validation
System metrics	psutil	CPU, memory, disk, network counters
Reachability	icmplib + OS fallback	Latency/loss abstraction
Packet/LAN features	Scapy (optional)	ARP and packet metadata collection
ML	scikit-learn (optional)	Lightweight Isolation Forest
Frontend	HTML/CSS/JS + Chart.js	Low RAM and simple deployment
Tests	pytest	Unit/integration testing
Formatting/linting	Ruff + Black	Consistent code quality

3. Process architecture

```
flowchart LR
  SCH[Scheduler] --> HC[Host Collector]
  SCH --> PC[Probe Collector]
  SCH --> NC[Optional Packet/Discovery Collector]
  HC --> BUS[Metric Service]
  PC --> BUS
  NC --> BUS
  BUS --> DB[(SQLite)]
  BUS --> AE[Alert Engine]
  AE --> DB
  API[FastAPI REST] --> DB
  WS[WebSocket Hub] --> BUS
  UI[Browser Dashboard] --> API
```

```
    UI --> WS
```

4. Runtime components

4.1 Scheduler

Runs collector jobs independently. A slow device probe must not block host metrics. Use asyncio tasks where practical; place blocking operations behind asyncio.to_thread() or equivalent.

4.2 Collectors

Collectors return typed domain measurements. They do not write directly to the database.

Recommended interfaces:

```
HostCollector.collect() -> HostSnapshot
NetworkIOCollector.collect() -> NetworkIOSnapshot
ProbeCollector.probe(device) -> ProbeResult
DiscoveryCollector.scan(subnet) -> list[DiscoveredDevice]
PacketCollector.sample(window_seconds) -> ProtocolSnapshot
```

4.3 Metric service

Validates, normalizes timestamps/units, persists measurements, updates latest-state cache, and publishes live events.

4.4 Alert engine

Consumes normalized observations and evaluates active rules. It owns deduplication, cooldown, recovery/resolution behavior, and severity assignment.

4.5 API layer

Routes only validate request/response and call services. Avoid database logic inside route functions.

4.6 WebSocket hub

Maintains connected clients and broadcasts compact dashboard snapshots. Do not stream every raw packet.

5. Polling defaults

- host resource metrics: 5 seconds;
- network I/O rates: 2-5 seconds;
- configured device status/latency: 10 seconds;
- packet-loss sample: 30 seconds or incorporated into device probe;
- alert evaluation: after each relevant normalized measurement;
- dashboard live broadcast: 2-5 seconds;
- cleanup/retention: once daily.

All intervals are configuration values.

6. Data units

- timestamps: UTC ISO-8601 in API; timezone-aware datetime internally;
- latency: milliseconds;
- packet loss: percentage 0-100;
- byte counters: bytes;

- rates: bits per second or Mbps at presentation layer;
- CPU/RAM/disk: percentage 0-100.

7. Concurrency rules

- One scheduler owns recurring jobs.
- Avoid starting duplicate scheduler instances during development auto-reload. Create lifecycle hooks carefully.
- SQLite writes should be short transactions.
- Limit concurrent probes with a semaphore (for example 10) to avoid burst load.

8. Configuration

Suggested environment variables:

```
NETWATCH_ENV=development
NETWATCH_HOST=127.0.0.1
NETWATCH_PORT=8000
NETWATCH_DB_URL=sqlite:///./data/netwatch.db
HOST_METRIC_INTERVAL=5
DEVICE_PROBE_INTERVAL=10
LIVE_PUSH_INTERVAL=3
METRIC_RETENTION_DAYS=30
ENABLE_DISCOVERY=false
ENABLE_PACKET_CAPTURE=false
ENABLE_ML=false
LOG_LEVEL=INFO
```

9. Error model

Domain/services should raise specific exceptions such as:

- ValidationError
- ProbeTimeoutError
- PermissionUnavailableError
- CollectorUnavailableError
- RepositoryError

API middleware converts expected exceptions into structured JSON errors.

10. Logging

Log structured events such as:

```
collector.host.success
collector.probe.timeout
alert.opened
alert.updated
alert.resolved
websocket.connected
retention.deleted
```

11. Health score design

Start deterministic and explainable.

Example score (0-100):

- Start at 100.
- Offline monitored critical device: -20 each, capped.
- Packet loss > 10%: -15.
- Packet loss 2-10%: -5.
- Average latency > 150 ms: -10.
- Average latency 75-150 ms: -5.
- Host CPU > 90% sustained: -5.
- Active critical alert: -10 each, capped.

Clamp to 0-100 and return a list of penalty reasons. Thresholds remain configurable.

12. Optional anomaly detection design

Features can include latency, packet loss, rx_mbps, tx_mbps, active_connections, CPU, memory, packets_per_second. Train only after enough baseline samples. Persist model metadata and training period. Anomaly score supplements alerts but does not automatically label security attacks.

13. Security boundaries

- Bind to localhost by default.
- Never accept arbitrary shell commands from API inputs.
- Validate IP/subnet inputs.
- Restrict scans to configured authorized ranges.
- Store no raw payloads by default.
- Keep secrets in .env, never commit them.
- Add authentication before binding the dashboard to untrusted networks.

Architecture and Module Design

1. Repository layout

```
netwatch/
├─ README.md
├─ pyproject.toml
├─ .env.example
├─ .gitignore
├─ docs/
│  └─ ... this documentation pack ...
├─ backend/
│  └─ app/
│     ├─ main.py
│     ├─ core/
│     │  ├─ config.py
│     │  ├─ logging.py
│     │  └─ lifecycle.py
│     ├─ api/
│     │  ├─ dependencies.py
│     │  └─ routes/
│     │     ├─ health.py
│     │     ├─ devices.py
│     │     ├─ metrics.py
│     │     ├─ alerts.py
│     │     ├─ settings.py
│     │     └─ websocket.py
│     ├─ collectors/
│     │  ├─ host.py
│     │  ├─ network_io.py
│     │  ├─ probe.py
│     │  ├─ discovery.py
│     │  └─ packet.py
│     ├─ services/
│     │  ├─ scheduler.py
│     │  ├─ metrics.py
│     │  ├─ alerts.py
│     │  ├─ health_score.py
│     │  ├─ discovery.py
│     │  └─ anomaly.py
│     ├─ db/
│     │  ├─ session.py
│     │  ├─ base.py
│     │  └─ models.py
│     ├─ repositories/
│     │  ├─ devices.py
│     │  ├─ metrics.py
│     │  ├─ alerts.py
│     │  └─ settings.py
│     ├─ schemas/
│     │  ├─ device.py
│     │  ├─ metric.py
│     │  ├─ alert.py
│     │  └─ dashboard.py
│     └─ utils/
│        ├─ time.py
│        └─ network.py
├─ frontend/
│  ├─ index.html
│  ├─ css/app.css
│  └─ js/
```

```
│     ├─ api.js
│     ├─ websocket.js
│     ├─ charts.js
│     └─ dashboard.js
├─ data/
│  └─ .gitkeep
├─ scripts/
│  ├─ seed_demo_devices.py
│  └─ export_metrics.py
└─ tests/
   ├─ unit/
   ├─ integration/
   └─ fixtures/
```

2. Dependency direction

Use this direction:

```
API / scheduler
      ↓
services
      ↓
collectors + repositories
      ↓
OS/network + database
```

Routes should not import concrete SQL statements. Collectors should not import FastAPI.

3. Main execution flow

```
sequenceDiagram
  participant S as Scheduler
  participant C as Collector
  participant M as Metric Service
  participant D as Database
  participant A as Alert Engine
  participant W as WebSocket Hub
  S->>C: collect/probe
  C-->>S: typed measurement
  S->>M: ingest(measurement)
  M->>D: persist
  M->>A: evaluate
  A->>D: open/update/resolve alert
  M->>W: publish latest summary
  W-->>W: broadcast to clients
```

4. Module responsibilities

core

Configuration, logging, startup/shutdown, global application concerns.

collectors

Interactions with operating system or network. Return clean data objects and never contain presentation logic.

services

Business rules: normalization, orchestration, health score, alert evaluation, anomaly scoring.

repositories

Database access only. Each repository exposes operations required by services.

schemas

Pydantic request/response/domain transport types.

api/routes

HTTP/WebSocket surface. Keep thin.

frontend

Renders dashboard and requests/streams data.

5. Dashboard information architecture

- Overview: health score, device counts, host metrics, bandwidth, active alerts, recent latency chart.
- Devices: table of configured/discovered devices with status and latest measurements.
- Device Detail: latency/loss history, state timeline, metadata, related alerts.
- Network: upload/download charts, protocol counters when enabled, connection summary.
- Alerts: filterable incident list, severity, acknowledge/resolve state.
- Reports: date-range summary and CSV export.
- Settings: polling, retention, thresholds, optional features.

6. Collector behavior

Each collector should expose:

- supported flag / readiness check;
- timeout;
- collection function;
- consistent result with timestamp;
- explicit error details without throwing unhandled exceptions.

7. Development rule

Every new feature should follow a vertical slice:

1. schema/domain object;

2. collector/service logic;

3. repository/database change;

4. API contract;

5. test;

6. UI representation;

7. logging and error state.

This prevents the UI, database, and backend from drifting apart.

Database and API Specification

1. Database conventions

- SQLite file: data/netwatch.db.
- All primary keys: integer or UUID; choose one convention and keep it consistent. For the MVP, integer primary keys are simplest.
- All timestamps: timezone-aware UTC.
- Use indexes on timestamps and foreign keys used in historical queries.
- Do not store high-frequency duplicate text; normalize device references.

2. Core tables

devices

Field	Type	Notes
id	INTEGER PK	Auto increment
name	TEXT	Required
ip_address	TEXT	Required, validated
hostname	TEXT NULL	Optional
mac_address	TEXT NULL	Optional
device_type	TEXT NULL	laptop/router/server/etc.
enabled	BOOLEAN	Default true
critical	BOOLEAN	Default false
probe_interval_s	INTEGER	Default from config
created_at	DATETIME	UTC
updated_at	DATETIME	UTC

device_metrics

Field	Type	Notes
id	INTEGER PK	
device_id	FK devices.id	Indexed
timestamp	DATETIME	Indexed
status	TEXT	online/offline/unknown
latency_ms	REAL NULL	
packet_loss_pct	REAL NULL	
probe_method	TEXT	icmp/etc.
error_code	TEXT NULL	timeout/permission/etc.

host_metrics

Field	Type	Notes
id	INTEGER PK	
timestamp	DATETIME	Indexed
cpu_pct	REAL	
memory_pct	REAL	
disk_pct	REAL	
active_connections	INTEGER NULL	
process_uptime_s	REAL	

network_io_metrics

Field	Type	Notes
id	INTEGER PK	
timestamp	DATETIME	Indexed
interface	TEXT	
bytes_sent_total	INTEGER	Cumulative counter
bytes_recv_total	INTEGER	Cumulative counter
packets_sent_total	INTEGER	
packets_recv_total	INTEGER	
tx_bps	REAL	Delta-derived
rx_bps	REAL	Delta-derived

alert_rules

Field	Type	Notes
id	INTEGER PK	
name	TEXT	
metric	TEXT	latency_ms/cpu_pct/etc.
operator	TEXT	gt/gte/lt/lte/eq
threshold	REAL	
duration_s	INTEGER	Must persist this long
severity	TEXT	info/warning/critical
enabled	BOOLEAN	

alerts

Field	Type	Notes
id	INTEGER PK	
rule_id	FK alert_rules.id NULL	
device_id	FK devices.id NULL	Host alerts may be null
source_key	TEXT	Dedup key
severity	TEXT	
status	TEXT	active/acknowledged/resolved
message	TEXT	
first_seen	DATETIME	
last_seen	DATETIME	
resolved_at	DATETIME NULL	
acknowledged_at	DATETIME NULL	
occurrence_count	INTEGER	

app_settings

key	value	Notes
TEXT PK	TEXT	Small editable runtime settings

3. Optional tables

- discovery_observations
- protocol_metrics
- anomaly_scores
- model_registry

4. REST API conventions

Base URL: /api/v1

Health

- GET /health - application health and feature readiness.

Dashboard

- GET /dashboard/summary - latest overview cards and health score.
- GET /dashboard/history?window=1h - compact chart data.

Devices

- GET /devices
- POST /devices
- GET /devices/{device_id}
- PATCH /devices/{device_id}

- DELETE /devices/{device_id} - soft-delete or disable is preferable for history.
- POST /devices/{device_id}/probe - one manual authorized probe.

Metrics

- GET /metrics/host?from=...&to=...&limit=...
- GET /metrics/network?interface=...&from=...&to=...
- GET /metrics/devices/{device_id}?from=...&to=...

Alerts

- GET /alerts?status=active&severity=critical
- POST /alerts/{alert_id}/acknowledge
- POST /alerts/{alert_id}/resolve - optional manual resolution.

Settings

- GET /settings
- PATCH /settings

Discovery (P1)

- POST /discovery/scan - requires explicit authorized subnet.
- GET /discovery/latest

Export

- GET /exports/metrics.csv?...

5. Example API payloads

POST /devices

```
{
  "name": "Home Router",
  "ip_address": "192.168.1.1",
  "device_type": "router",
  "critical": true,
  "enabled": true
}
```

Dashboard summary response

```
{
  "generated_at": "2026-09-28T18:00:00Z",
  "health_score": 91,
  "health_reasons": ["packet loss elevated on Home Router"],
  "devices": {"total": 4, "online": 3, "offline": 1, "unknown": 0},
  "host": {"cpu_pct": 31.2, "memory_pct": 59.4, "disk_pct": 52.1},
  "network": {"rx_mbps": 7.3, "tx_mbps": 1.1},
  "active_alerts": 2
}
```

6. WebSocket contract

Endpoint: /ws/live

Recommended message envelope:

```
{
  "type": "dashboard.snapshot",
  "timestamp": "2026-09-28T18:00:03Z",
```

```
  "data": {}
}
```

Other message types:

- device.updated
- alert.opened
- alert.updated
- alert.resolved
- health_score.updated

The dashboard should still be able to load all state through REST after reconnecting; WebSocket messages are an optimization, not the only source of truth.

7. Retention strategy

Start with 30 days of raw metrics. A daily cleanup job deletes older rows. If longer retention is needed later, aggregate older data into 1-minute or 5-minute buckets before deletion.

Implementation Roadmap

Development rule

Do not work on multiple unfinished phases simultaneously. Each phase ends with a runnable application and explicit acceptance gate.

Phase 0 - Repository and environment

Tasks

- Create repository/folder structure.
- Create virtual environment.
- Add dependency management (pyproject.toml or requirements files).
- Add .env.example, .gitignore, formatter/linter/test configuration.
- Create minimal FastAPI app with /api/v1/health.
- Add structured logging.

Exit criteria

- uvicorn backend.app.main:app --reload starts successfully.
- Health endpoint returns 200.
- pytest runs.
- ruff check/formatter passes.

Phase 1 - Local telemetry

Tasks

- Implement host CPU/memory/disk collector.
- Implement network I/O counter collector.
- Correctly calculate byte-rate deltas using elapsed time.
- Add typed snapshots and unit tests.
- Display latest values in terminal/logs before adding database.

Exit criteria

- Values update every configured interval.
- No negative rate after normal counter changes; counter resets are handled.
- Collector failures are logged without stopping the scheduler.

Phase 2 - Persistence

Tasks

- Create SQLAlchemy models and session.
- Add repository layer.
- Persist host and network I/O metrics.
- Add retention setting.
- Add query by date range.

Exit criteria

- Restarting app preserves history.
- Queries return ordered measurements.

- Test database is isolated from development database.

Phase 3 - Device monitoring

Tasks

- Device CRUD.
- Probe collector with timeout.
- Latency measurement.
- Packet-loss sampling.
- Online/offline/unknown state.
- Concurrent probe limit.

Exit criteria

- Router/localhost/test fixture can be monitored.
- Unreachable IP does not freeze application.
- Device status is stored and queryable.

Phase 4 - REST API

Tasks

- Implement device, metric, dashboard, and alert endpoints.
- Add response schemas.
- Add query validation and reasonable limits.
- Verify OpenAPI docs.

Exit criteria

- Critical flows work from Swagger UI/curl.
- Invalid inputs return 4xx with clean messages.
- No raw stack traces reach client.

Phase 5 - Dashboard MVP

Tasks

- Build static dashboard shell.
- Overview cards.
- Device table.
- Chart.js time-series charts.
- Device detail page/panel.
- Loading, empty, and error states.

Exit criteria

- Dashboard loads current data from API.
- Charts render historical series.
- At 1366x768 no critical content overlaps.

Phase 6 - Live updates

Tasks

- Add WebSocket hub.
- Broadcast compact snapshot.
- Frontend reconnect strategy.
- REST resync after reconnect.

Exit criteria

- Live cards/charts update without page refresh.
- Disconnect/reconnect does not duplicate listeners or crash backend.

Phase 7 - Alert engine

Tasks

- Implement rules and comparator abstraction.
- Add duration/debounce logic.
- Add deduplication and recovery.
- Add acknowledge endpoint/UI.
- Seed sensible default rules.

Exit criteria

- Controlled threshold breach creates one active incident.
- Continuing breach updates incident count/last_seen.
- Recovery resolves incident.
- Alert timeline is visible in UI.

Phase 8 - Network health score and reports

Tasks

- Implement deterministic score and reason list.
- Add daily/selected-period summary.
- CSV export.

Exit criteria

- Score changes can be explained from source metrics/alerts.
- CSV opens correctly in spreadsheet tools.

Phase 9 - LAN discovery (optional P1)

Tasks

- Add explicit subnet input.
- Validate private/authorized subnet.
- Implement ARP scan with Scapy where supported.
- Show discovered-but-unmanaged devices separately.
- Allow user to promote a discovery result to monitored device.

Exit criteria

- Feature fails gracefully without privilege/Npcap.
- Core monitoring remains functional when discovery is disabled.

Phase 10 - Packet/protocol metrics (optional P1)

Tasks

- Capture metadata only for a short sample window.
- Count TCP/UDP/ICMP/DNS or other simple categories.
- Store aggregates, not payloads.
- Display protocol distribution.

Exit criteria

- Permission error is clear.
- No packet payload persistence by default.

Phase 11 - Anomaly detection (optional P2)

Tasks

- Define feature vector and minimum baseline size.
- Build dataset from existing database.
- Train Isolation Forest offline/in-process on bounded samples.
- Persist model metadata.
- Score new windows.
- Display anomaly score and contributing raw metric deviations.

Exit criteria

- ML can be fully disabled.
- Deterministic monitoring remains authoritative.
- Evaluation includes false-positive observations and baseline limitations.

Recommended milestone schedule

- Milestone A: Phases 0-2 - data collection and storage.
- Milestone B: Phases 3-5 - complete monitoring dashboard.
- Milestone C: Phases 6-8 - real-time alerts and reporting.
- Milestone D: Phases 9-11 - advanced networking/ML extensions.

Stop condition

If a phase is unstable, do not add the next advanced feature. Fix tests, resource usage, and data correctness first.

Testing and Acceptance Plan

1. Test levels

- Unit tests: pure calculations, validation, rule evaluation, rate calculation.
- Collector tests: mock OS/network responses.
- Repository tests: temporary SQLite database.
- API integration tests: FastAPI test client.
- WebSocket tests: connection, payload, reconnect behavior.
- Manual system tests: real laptop/router conditions.
- Performance sanity tests: CPU/RAM/database growth over a sustained run.

2. Critical unit tests

Network I/O rate

- normal counter increase;
- zero elapsed time guard;
- counter reset/wrap;
- interface disappears;
- large burst.

Health score

- perfect network = 100;
- penalties apply exactly once;
- score clamps to 0;
- reasons correspond to penalties.

Alert engine

- threshold not reached;
- threshold reached;
- required duration not yet reached;
- incident opens once;
- repeated breach updates same incident;
- condition recovers and incident resolves;
- acknowledged incident remains traceable.

Validation

- valid IPv4;
- invalid IP rejected;
- invalid polling interval rejected;
- unsafe/unsupported subnet rejected for discovery.

3. Integration tests

- collector -> metric service -> database.
- device probe -> persistence -> dashboard summary.
- metric breach -> alert creation -> API output.
- database restart -> historical data still available.

- WebSocket snapshot contains latest persisted/derived values.

4. Manual acceptance scenarios

AT-01 Normal host monitoring

Given NetWatch is running for 2 minutes

When the dashboard is open

Then CPU, memory, disk, RX and TX values update and history contains multiple samples.

AT-02 Device online

Add the local router or another authorized reachable device. Verify online state, latency, and timestamps update.

AT-03 Device offline

Temporarily monitor a known unused private IP or disconnect an owned test device. Verify timeout handling, offline/unknown state, and alert behavior without backend freeze.

AT-04 High local load

Generate benign local CPU load for a short controlled period. Verify high-CPU alert opens after configured duration and resolves when load returns below recovery threshold.

AT-05 Packet-loss simulation

Use a controlled test environment or a mocked collector. Verify packet-loss threshold logic and UI message.

AT-06 WebSocket reconnect

Stop browser network temporarily or restart backend. Verify the page reconnects/resynchronizes without requiring full browser restart.

AT-07 Retention

Insert/seed expired test records. Run retention job and confirm only out-of-retention rows are deleted.

AT-08 CSV export

Export a time range and verify column headings, ISO timestamps, numeric values, and row count.

5. Performance targets for student-laptop deployment

- Base memory: aim < 1 GB.
- Sustained CPU: aim < 10% during normal light monitoring, recognizing platform differences.
- Dashboard summary API: aim < 500 ms for local use.
- Initial page load: aim < 3 seconds on the same laptop.
- Device count target for MVP: 1-20 monitored endpoints.

These are engineering targets, not absolute guarantees; record actual measurements in the final report.

6. Test data strategy

Use three categories:

1. Real original data: normal laptop/network telemetry.

2. Controlled conditions: disconnect/reconnect owned devices, local load, permitted lab traffic.

3. Mocked fixtures: deterministic edge cases that are difficult or unsafe to reproduce physically.

Do not create disruptive traffic against networks or devices you do not own or have explicit permission to test.

7. Definition of Done for every feature

A feature is done only when:

- requirements/contract are clear;
- code is modular;
- unit/integration tests pass where applicable;
- errors are handled;
- logs are meaningful;
- UI has loading/empty/error states if exposed visually;
- documentation is updated;
- no secret or local database is accidentally committed.

Setup and Runbook

1. Development prerequisites

- Windows 10/11 or a modern Linux distribution.
- Python 3.11 or newer recommended.
- Git.
- VS Code/PyCharm/Cursor or another Python-capable IDE.
- Browser.
- Optional: Npcap on Windows for Scapy packet capture/ARP features.

2. Environment setup

From the project root:

Windows PowerShell

```
py -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -e .[dev]
copy .env.example .env
```

Linux/macOS shell

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -e '.[dev]'
cp .env.example .env
```

If the project uses requirements.txt instead, replace the editable install command with pip install -r requirements.txt.

3. Suggested dependencies

Core:

```
fastapi
uvicorn[standard]
sqlalchemy
pydantic-settings
psutil
icmplib
```

Development:

```
pytest
pytest-asyncio
httpx
ruff
black
```

Optional:

```
scapy
scikit-learn
joblib
```

4. Start command

```
uvicorn backend.app.main:app --reload --host 127.0.0.1 --port 8000
```

Open:

- Dashboard: http://127.0.0.1:8000/
- API docs: http://127.0.0.1:8000/docs
- Health: http://127.0.0.1:8000/api/v1/health

5. IDE configuration

- Select .venv as the Python interpreter.
- Enable format-on-save with Ruff/Black if desired.
- Use project root as working directory.
- Store environment values in .env, not IDE-specific hard-coded paths.
- Add a run configuration for Uvicorn and a second one for pytest.

6. Privileged features

Windows

Scapy packet capture or ARP discovery may require:

- Npcap installed;
- IDE/terminal launched with Administrator rights.

Linux

Raw sockets may require root/capabilities. Prefer running only the specific privileged collector with the minimum permissions necessary rather than the whole IDE as root.

The application must print a clear feature-disabled warning when privileges are absent.

7. Common troubleshooting

`ModuleNotFoundError`

Confirm IDE interpreter is .venv and install dependencies inside that environment.

Ping reports unreachable but device is online

The device/firewall may block ICMP. Treat this as probe-specific unknown rather than assuming the device is powered off. Later add an optional TCP/service probe.

Scapy permission error

Disable packet capture/discovery and continue with core monitoring, or configure required OS permissions.

SQLite `database is locked`

Keep transactions short, avoid excessive simultaneous write sessions, and use one well-managed session pattern per task/request.

Duplicate collectors when using `--reload`

Ensure scheduler startup is tied correctly to application lifespan and is not launched by module import side effects.

Network speed shows cumulative bytes

Calculate difference between two counter readings divided by elapsed seconds. Never display raw cumulative bytes as Mbps.

8. Backup and reset

- Database location: data/netwatch.db.
- To preserve history, copy the database while the app is stopped.
- To reset during early development, stop the app and remove the development database only if migrations/data are not needed.

9. Git hygiene

Do not commit:

```
.venv/
.env
data/*.db
__pycache__/
.pytest_cache/
.ruff_cache/
models/*.joblib
```

Commit .env.example with safe defaults and no passwords/tokens.

IDE Execution Guide

How to use this file

Work top-to-bottom. For each task, implement only the specified scope, run tests, and satisfy the acceptance check before moving forward. This file can also be supplied to an IDE coding assistant as project context.

Global engineering rules

1. Keep route handlers thin.

2. Do not put all logic in main.py.

3. Use type hints for public functions.

4. Use UTC internally.

5. Every network operation has an explicit timeout.

6. Never crash the scheduler because one device fails.

7. Never persist raw packet payloads by default.

8. Do not start ML until real telemetry storage and alerting are stable.

9. Write tests for calculations and alert state transitions before relying on UI behavior.

10. Prefer simple code over unnecessary microservices or framework layers.

Backlog

NW-001 Bootstrap backend

Objective: runnable FastAPI service.

Implement: application factory/lifespan, config, logging, /api/v1/health.

Acceptance: service starts; health endpoint returns version/environment/database readiness; pytest smoke test passes.

NW-002 Host collector

Objective: collect original laptop CPU, RAM, disk, uptime.

Implement: psutil collector returning typed snapshot.

Acceptance: values are within valid ranges; exceptions produce structured failure, not crash.

NW-003 Network I/O collector

Objective: original RX/TX telemetry.

Implement: per-interface counters + delta rate calculator.

Acceptance: Mbps changes as traffic changes; reset case is handled.

NW-004 Persistence layer

Objective: durable telemetry.

Implement: SQLAlchemy session/models/repositories and SQLite initialization.

Acceptance: samples remain after restart; tests use temporary DB.

NW-005 Scheduler

Objective: recurring non-blocking collection.

Implement: host and network jobs with clean shutdown.

Acceptance: no duplicate tasks; exceptions are isolated; intervals configurable.

NW-006 Device CRUD

Objective: register monitored devices.

Implement: device model, validation, CRUD service/API.

Acceptance: valid private/local addresses can be saved; invalid values rejected.

NW-007 Reachability/latency collector

Objective: monitor authorized devices.

Implement: timed probe, latency, state mapping.

Acceptance: reachable and unreachable cases return quickly within timeout.

NW-008 Packet-loss sampling

Objective: quantify instability.

Implement: multiple small probes per sampling cycle or supported library loss measurement.

Acceptance: loss percentage 0-100; timeout behavior tested.

NW-009 Metrics APIs

Objective: current/history access.

Implement: bounded date-range endpoints.

Acceptance: ordered JSON series; invalid ranges rejected; limit enforced.

NW-010 Dashboard shell

Objective: usable UI.

Implement: overview cards, navigation, loading/error states, API client.

Acceptance: no hard-coded fake metrics once live API is connected.

NW-011 Charts

Objective: historical visualization.

Implement: Chart.js latency, loss, RX/TX, CPU/RAM charts.

Acceptance: correct axes/units; empty data handled.

NW-012 Live WebSocket updates

Objective: no page refresh.

Implement: hub, snapshot message, frontend reconnect.

Acceptance: page updates live and recovers after backend restart.

NW-013 Alert engine

Objective: deterministic incidents.

Implement: rule evaluation, duration, dedup, recovery.

Acceptance: one incident per continuous violation; resolution recorded.

NW-014 Alert UI

Objective: actionable dashboard.

Implement: active/history list, severity, acknowledgement.

Acceptance: acknowledgement persists and visible after refresh.

NW-015 Health score

Objective: summarized network status.

Implement: deterministic 0-100 score plus reason list.

Acceptance: unit tests cover each penalty and score is explainable.

NW-016 CSV export

Objective: project/report data reuse.

Implement: streamed/bounded CSV response.

Acceptance: selected time range exported with correct units/timestamps.

NW-017 Retention

Objective: bounded database size.

Implement: daily cleanup.

Acceptance: only records older than policy are removed.

NW-018 LAN discovery (optional)

Objective: discover authorized local devices.

Implement: explicit subnet scan with Scapy and privilege checks.

Acceptance: result can be added to monitoring; failure does not affect core app.

NW-019 Protocol aggregation (optional)

Objective: traffic composition.

Implement: short-window packet metadata counter.

Acceptance: aggregate counts only; no payload storage.

NW-020 Anomaly baseline (optional)

Objective: detect unusual metric combinations.

Implement: bounded historical feature extraction + Isolation Forest.

Acceptance: feature is switchable, model metadata stored, results labelled as anomaly score rather than attack classification.

Suggested coding-assistant task prompt template

Use one ticket at a time:

```
Implement ticket NW-00X for the NetWatch project.
Follow docs/02_SRS.md, docs/03_TRD.md, docs/04_ARCHITECTURE_AND_MODULES.md and
docs/05_DATABASE_AND_API_SPEC.md.
Do not implement later tickets.
Add/update automated tests for the ticket.
Keep the application runnable.
At the end, report files changed, commands to run, tests executed, and any assumptions.
```

Commit strategy

Prefer one logical ticket per commit:

```
feat(collector): add host metrics [NW-002]
feat(db): persist host and network metrics [NW-004]
feat(alerts): add incident deduplication [NW-013]
```

Daily development checklist

- Pull/open correct branch.
- Activate .venv.
- Run tests before changes.
- Implement one ticket.
- Run targeted tests, then full tests.
- Run lint/format.
- Launch app and manually verify the changed flow.
- Update docs/API contract if behavior changed.
- Commit only working code.

Academic Report and Demonstration Guide

1. Suggested project title

NetWatch: Real-Time Network Monitoring, Performance Analysis and Anomaly Detection System

2. Suggested abstract

NetWatch is a lightweight real-time network monitoring platform designed for small networks and resource-constrained development environments. The system collects original telemetry from the host computer and authorized network devices, including CPU and memory utilization, network input/output rates, device availability, latency, and packet loss. A FastAPI backend processes and stores the measurements in a local SQLite database, while a web dashboard provides live status, historical charts, network-health scoring, and threshold-based alerts. The architecture is modular so optional capabilities such as LAN discovery, protocol aggregation, SNMP monitoring, and Isolation Forest-based anomaly detection can be added without changing the core monitoring pipeline. The project emphasizes reproducible real-time data collection, explainable alerting, low resource usage, and practical network observability on a standard laptop.

3. Report chapter outline

1. Introduction

2. Problem statement

3. Objectives

4. Literature/technology background

5. Requirement analysis

6. System architecture

7. Module design

8. Database design

9. Implementation

10. Real-time data collection methodology

11. Alert and health-score logic

12. Testing methodology

13. Results and performance measurements

14. Optional anomaly-detection experiment

15. Limitations

16. Future scope

17. Conclusion

18. References

4. Evidence to collect during implementation

Save screenshots/tables for:

- application dashboard under normal state;
- device going offline and recovering;
- latency/packet-loss chart;
- CPU/RAM/network traffic chart;
- active alert and resolved alert;
- Swagger/OpenAPI endpoints;
- SQLite sample rows or export;
- resource usage of NetWatch itself;
- optional discovery/protocol/anomaly view.

5. Evaluation metrics

For monitoring system:

- average collection interval accuracy;
- API response time;
- dashboard refresh delay;
- CPU/RAM used by NetWatch;
- database growth per hour/day;
- uptime during sustained test.

For device monitoring:

- probe success/failure handling;
- measured latency distribution;
- packet-loss observations;
- time to reflect state change.

For optional anomaly detection:

- baseline duration/sample count;
- anomaly-rate during normal period;
- anomalies during controlled test conditions;
- false positives and limitations.

6. Demonstration script (10-15 minutes)

1. Explain problem and architecture (1-2 min).

2. Start application and show health/API docs (1 min).

3. Open dashboard and explain original live data (2 min).

4. Show monitored router/authorized device latency and loss (2 min).

5. Generate safe controlled condition, such as temporarily disconnecting an owned device or CPU load, and show alert lifecycle (2-3 min).

6. Show historical charts and CSV export (2 min).

7. Show optional discovery/anomaly module if completed (1-2 min).

8. End with resource usage, limitations, and future work (1 min).

7. Claims to avoid

Do not claim that an Isolation Forest anomaly is necessarily a cyberattack. Do not claim a device is powered off solely because ICMP is blocked. Distinguish measurements, inferred state, and optional model output.

8. Future-scope ideas

- SNMPv3 monitoring;
- secure remote agents;
- Prometheus-compatible exporter;
- topology graph;
- notification integrations;
- role-based authentication;
- PostgreSQL/TimescaleDB for larger deployments;
- more rigorous anomaly evaluation;
- deployment on a Raspberry Pi or lab server.